

Java Persistence API (JPA), Hibernate, and Spring Data JPA Explained

1. Java Persistence API (JPA)

JPA stands for Java Persistence API. It is a specification (JSR 338) provided by Java that defines a standard way to manage relational database operations using Java objects.

Important points:

- JPA is only a specification, not a tool or framework.
- It defines interfaces, annotations, and rules for Object Relational Mapping (ORM).
- JPA itself cannot communicate with the database because it has no implementation.
- Developers use a JPA implementation such as Hibernate, EclipseLink, or OpenJPA.

Common JPA annotations:

- `@Entity`: Marks a Java class as a database entity.
- `@Id`: Defines the primary key.
- `@Table`: Maps an entity to a database table.
- `@Column`: Maps a field to a database column.

Example:

`@Entity`

```
public class Student {
```

```
    @Id
```

```
    private Long id;
```

```
    private String name;
```

```
}
```

2. Hibernate

Hibernate is an ORM (Object Relational Mapping) framework and one of the most popular implementations of JPA.

Hibernate provides the actual functionality required to interact with databases.

Features:

- Converts Java objects into database records.

- Converts database records into Java objects.
- Handles SQL generation automatically.
- Provides caching, lazy loading, and transaction management.
- Reduces manual JDBC code.

Relationship:

JPA = Rules/Specification

Hibernate = Implementation that follows JPA rules

Example:

When an application calls a JPA method, Hibernate performs the actual SQL operations behind the scenes.

3. Spring Data JPA

Spring Data JPA is a module of the Spring Framework that provides an abstraction layer over JPA implementations like Hibernate.

Its main goal is to reduce boilerplate code required for database operations.

Without Spring Data JPA:

Developers need to create DAO classes, write queries, manage EntityManager, and handle common CRUD operations manually.

With Spring Data JPA:

Developers can create repository interfaces and get CRUD functionality automatically.

Example:

```
public interface StudentRepository extends JpaRepository {  
  
}
```

This automatically provides:

- save()
- findById()
- findAll()
- delete()
- update operations

4. Difference Between JPA, Hibernate, and Spring Data JPA

JPA:

- Type: Specification
- Purpose: Defines standards for persistence
- Provides: Interfaces and annotations
- Does not execute database operations

Hibernate:

- Type: ORM Framework
- Purpose: Implements JPA specification
- Provides: Database communication, SQL generation, caching

Spring Data JPA:

- Type: Spring abstraction framework
- Purpose: Simplifies JPA usage
- Provides: Repository pattern and ready-made CRUD methods

5. Architecture Flow

Application Code

|

v

Spring Data JPA Repository

|

v

JPA Specification

|

v

Hibernate Implementation

|

v

Database

6. Spring Data JPA vs Hibernate

Hibernate:

- Direct ORM framework.
- Requires more configuration and code.
- Provides advanced ORM features.

Spring Data JPA:

- Built on top of JPA and Hibernate.
- Removes repetitive code.
- Provides repository interfaces.
- Faster development for CRUD applications.

7. Summary

JPA defines what should be done.

Hibernate defines how database operations are performed.

Spring Data JPA makes working with JPA and Hibernate easier by reducing code.

In a typical Spring Boot application:

Spring Data JPA -> Hibernate -> Database